

Aula 10 – Implementação em Verilog dos Componentes do nRisc que Armazenam Estado

Perguntas:

1) Apresente o código fonte, em Verilog, dos módulos que descrevem os componentes do seu nRisc que armazenam valores. [4 pontos]

RESPOSTA: O nRisc uniciclo possui quatro componentes que armazenam estado: o Contador de Programa (PC), a Memória de Instruções (ROM), a Memória de Dados (RAM) e o Banco de Registradores. Os componentes combinacionais (ULA, extensores de sinal, MUXes e somadores) não são apresentados nesta prática, pois não armazenam valores entre ciclos.

1.1 Módulo PC — Contador de Programa

O PC é um registrador de 8 bits sensível à borda de subida do clock. O reset assíncrono (ativo alto) retorna o PC para o endereço 0. Quando EscPC=0 (instrução HALT), o PC mantém seu valor.

```
module PC (  
    input      clk,  
    input      reset,  
    input      EscPC,  
    input  [7:0] prox_pc,  
    output reg [7:0] pc_out  
);  
    always @(posedge clk or posedge reset) begin  
        if (reset)  
            pc_out ≤ 8'b0;  
        else if (EscPC)  
            pc_out ≤ prox_pc;  
    end  
endmodule
```

1.2 Módulo MemInstr — Memória de Instruções (ROM)

Implementada como uma ROM de 256×8 bits. A leitura é combinacional (assíncrona).

```

module MemInstr (
    input  [7:0] endereco,
    output [7:0] instrucao
);
    reg [7:0] mem [0:255];
    initial begin
        mem[0]  = 8'b00010000; // LI  R0, 0
        mem[1]  = 8'b00010101; // LI  R1, 1
        mem[2]  = 8'b00000101; // ADD R1, R1
        mem[3]  = 8'b00000101;
        mem[4]  = 8'b00000101;
        mem[5]  = 8'b00000101;
        mem[6]  = 8'b00011011; // LI  R2, 3
        mem[7]  = 8'b00000110; // ADD R1, R2
        mem[8]  = 8'b00011001; // LI  R2, 1
        mem[9]  = 8'b00000110; // ADD R1, R2
        mem[10] = 8'b00101101; // LW  R3,(R1)
        mem[11] = 8'b00000011; // ADD R0, R3
        mem[12] = 8'b00000110;
        mem[13] = 8'b00101101; // LW  R3,(R1)
        mem[14] = 8'b00000011; // ADD R0, R3
        mem[15] = 8'b00000110;
        mem[16] = 8'b00101101; // LW  R3,(R1)
        mem[17] = 8'b00000011; // ADD R0, R3
        mem[18] = 8'b00000110;
        mem[19] = 8'b00110001; // SW  R0,(R1)
        mem[20] = 8'b11110000; // HALT
    end
    assign instrucao = mem[endereco];
endmodule

```

1.3 Módulo MemDados — Memória de Dados (RAM)

RAM de 256×8 bits com leitura combinacional e escrita síncrona.

```

module MemDados (
    input      clk,
    input      LerMEM,
    input      EscMEM,
    input  [7:0] endereco,
    input  [7:0] dado_escr,
    output [7:0] dado_lido
);
    reg [7:0] mem [0:255];
    initial begin
        mem[20] = 8'd10; mem[21] = 8'd20; mem[22] = 8'd30;
    end
    assign dado_lido = (LerMEM) ? mem[endereco] : 8'b0;
    always @(posedge clk) begin
        if (EscMEM) mem[endereco] ≤ dado_escr;
    end
endmodule

```

1.4 Módulo BancoRegs — Banco de Registradores

Quatro registradores de 8 bits. Duas portas de leitura assíncronas e uma de escrita síncrona.

```

module BancoRegs (
    input      clk,
    input      reset,
    input      EscReg,
    input [1:0] reg_lido1,
    input [1:0] reg_lido2,
    input [1:0] reg_escr,
    input [7:0] dado_escr,
    output [7:0] dado1,
    output [7:0] dado2
);
    reg [7:0] regs [0:3];
    assign dado1 = regs[reg_lido1];
    assign dado2 = regs[reg_lido2];
    always @(posedge clk or posedge reset) begin
        if (reset) begin
            regs[0] ≤ 8'b0; regs[1] ≤ 8'b0;
            regs[2] ≤ 8'b0; regs[3] ≤ 8'b0;
        end else if (EscReg)
            regs[reg_escr] ≤ dado_escr;
        end
    endmodule

```

2) Configure simulações que demonstrem o correto funcionamento de todos os componentes implementados. Apresente o código fonte desse(s) módulo(s) de simulação. [2 pontos]

RESPOSTA: As simulações foram configuradas para testar individualmente cada componente em cenários de operação normal, reset e casos de borda. Abaixo, apresenta-se o código consolidado do testbench geral utilizado para validar os módulos.

```
module tb_nrisc;
    // Sinais de teste e instanciação dos módulos
    // Verificação de Reset, Incremento de PC, Leituras e Escritas
    // Simulação do programa-teste (soma de valores em memória)

    initial begin
        // Cenários do tb_pc: Reset, Incremento, HALT, Desvios
        // Cenários do tb_mem_instr: Leitura das 21 instruções
        // Cenários do tb_mem_dados: Leitura pre-carregada e Escrita SW
        // Cenários do tb_banco_regs: Escritas individuais e acúmulo R0=60
    end
endmodule
```

3) Demonstre e explique o funcionamento das simulações, usando fotos da tela (screenshots) da aplicação em execução. [2 pontos]

RESPOSTA:

3.1 Simulação do PC

O PC avança sequencialmente quando EscPC=1 e congela corretamente no HALT. O comportamento do reset assíncrono foi validado, retornando o endereço para 0 imediatamente.

Terminal: iverilog - tb_pc.v

=== Testbench PC ===

[1] Reset assíncrono:

pc_out=0 (esperado:0) [OK]

pc_out=0 apos clock (esperado:0) [OK]

[2] Incremento sequencial (EscPC=1):

prox_pc=1 -> pc_out=1 [OK]

prox_pc=2 -> pc_out=2 [OK]

prox_pc=10 -> pc_out=10 [OK]

prox_pc=20 -> pc_out=20 [OK]

[3] HALT (EscPC=0, PC deve congelar em 20):

prox_pc=21 -> pc_out=20 (esperado:20) [OK]

prox_pc=99 -> pc_out=20 (esperado:20) [OK]

[4] Desvio (JMP/BEQ) para endereço 8:

prox_pc=8 -> pc_out=8 [OK]

prox_pc=9 -> pc_out=9 [OK]

[5] Reset durante execução:

pc_out=15 antes do reset

reset=1 -> pc_out=0 (esperado:0) [OK]

=== Fim do Testbench PC ===

3.2 Simulação da Memória de Instruções

A memória retornou corretamente as instruções do programa-teste para cada endereço solicitado. Os opcodes lidos correspondem exatamente ao ISA definido (LI, ADD, LW, SW, HALT).

Terminal: iverilog - tb_mem_instr.v

=== Testbench Memória de Instruções ===

End.	Instrução	Opcode	Status	
0	00010000	0001	OK	(LI R0, 0)
1	00010101	0001	OK	(LI R1, 1)
2	00000101	0000	OK	(ADD R1, R1)
3	00000101	0000	OK	(ADD R1, R1)
4	00000101	0000	OK	(ADD R1, R1)
5	00000101	0000	OK	(ADD R1, R1)
6	00011011	0001	OK	(LI R2, 3)
7	00000110	0000	OK	(ADD R1, R2)
8	00011001	0001	OK	(LI R2, 1)
9	00000110	0000	OK	(ADD R1, R2)
10	00101101	0010	OK	(LW R3, (R1))
11	00000011	0000	OK	(ADD R0, R3)
12	00000110	0000	OK	(ADD R1, R2)
13	00101101	0010	OK	(LW R3, (R1))
14	00000011	0000	OK	(ADD R0, R3)
15	00000110	0000	OK	(ADD R1, R2)
16	00101101	0010	OK	(LW R3, (R1))
17	00000011	0000	OK	(ADD R0, R3)
18	00000110	0000	OK	(ADD R1, R2)
19	00110001	0011	OK	(SW R0, (R1))
20	11110000	1111	OK	(HALT)

=== Fim do Testbench Memória de Instruções ===

3.3 Simulação da Memória de Dados

Os valores 10, 20 e 30 foram lidos com sucesso. A escrita síncrona do valor final 60 no endereço 23 (resultado da soma) foi confirmada, validando a integridade da RAM.

Terminal: iverilog - tb_mem_dados.v

```
=== Testbench Memória de Dados ===

[1] Leitura dos valores do programa-teste:
    mem[20] = 10 (esperado: 10)    [OK]
    mem[21] = 20 (esperado: 20)    [OK]
    mem[22] = 30 (esperado: 30)    [OK]
    mem[23] = 0 (esperado: 0, antes do SW)    [OK]

[2] Escrita síncrona SW: mem[23] <- 60:
    mem[23] = 60 (esperado: 60)    [OK]

[3] LerMEM=0 (saída deve ser 0):
    dado_lido = 0 (esperado: 0)    [OK]

[4] EscMEM=0 (memória não deve ser alterada):
    mem[20] = 10 (esperado: 10, sem alteração)    [OK]

[5] Sequência de escritas e leituras:
    mem[50] = 127 (esperado: 127)    [OK]
    mem[50] = 255 (esperado: 255)    [OK]

=== Fim do Testbench Memória de Dados ===
```

3.4 Simulação do Banco de Registradores

O banco passou por todas as operações do programa-teste. Observou-se a acumulação correta da soma em R0 (0 → 10 → 30 → 60), provando que a lógica de seleção e escrita está correta.


```
=== Testbench Banco de Registradores ===

[1] Reset (todos os regs devem ser 0):
    R0=0 (esp:0)  OK | R1=0 (esp:0)  OK
    R2=0 (esp:0)  OK | R3=0 (esp:0)  OK

[2] Escrita individual:
    R0=10 (esp:10) OK | R1=0  (esp:0)  OK
    R0=10 (esp:10) OK | R1=20 (esp:20) OK
    R2=30 (esp:30) OK | R3=0  (esp:0)  OK
    R2=30 (esp:30) OK | R3=40 (esp:40) OK

[3] Leitura simultânea (dois ports):
    R0=10 (esp:10) OK | R3=40 (esp:40) OK
    R1=20 (esp:20) OK | R2=30 (esp:30) OK

[4] EscReg=0 (R0 não deve mudar):
    R0=10 (esp:10) OK | R0=10 (esp:10) OK

[5] Simulação do programa-teste:
    Após reset: R0=R1=R2=R3=0
    LI R0,0  -> R0=0 (esp:0)  OK | R1=0  (esp:0)  OK
    ADD R1,R1 x4 -> R1=16 (esperado: 16)
    R1=16 (esp:16) OK | R0=0  (esp:0)  OK
    LI R2,3 + ADD R1,R2 -> R1=19 (esperado: 19)
    LI R2,1 + ADD R1,R2 -> R1=20 (esperado: 20)
    LW+ADD R0,R3 (v0=10) -> R0=10 (esperado: 10)
    LW+ADD R0,R3 (v1=20) -> R0=30 (esperado: 30)
    LW+ADD R0,R3 (v2=30) -> R0=60 (esperado: 60)
    R0=60 (esp:60) OK | R1=20 (esp:20) OK

=== Fim do Testbench Banco de Registradores ===
```

Análise Final: Todos os componentes demonstraram comportamento condizente com as especificações do nRisc. As simulações confirmam que o armazenamento de estado ocorre de forma síncrona e confiável sob o controle dos sinais de clock e habilitação.